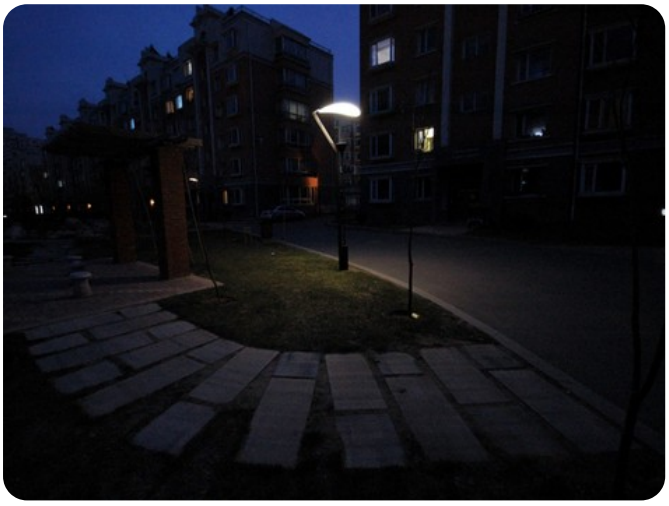
resenh

Robust low-light image restoration and enhancement

Technical solution report. Two methods restore each photo, one classical and one learned, and a controller decides how much of each to keep.



Input, as captured



The same photo after resenh, about 6 s in a laptop browser

Problem ID	P1	
Problem title	Robust Low-Light Image Restoration and Enhancement	
Team	V-Max (T46), section E	
Team members	Sharadh Naidu `1RUA24CSE0421` S. V. Swamy `1RUA24CSE0468`	Shivani `1RUA24CSE0430` Sumukha K `1RUA24CSE0483`
GitHub repository	github.com/SharadhNaidu/resen	
Live demo	resenh.sharadh.blog runs in the browser. The package is on PyPI, and a Colab notebook shows it in use.	

1 Problem understanding and solution strategy

Core technical challenge. A photo taken in the dark holds little light and a lot of sensor noise. Brightening it raises the noise too, pulls colours off true, burns lamps and windows to white, and overexposes any part of the scene that was already lit. No single fixed curve gets all of this right, because each photo needs its own amount of gain, denoising and contrast.

We also set three constraints. The result has to arrive within 10 seconds on an ordinary laptop or phone, without a server. The evaluation has to be reproducible, using plain PSNR with nothing tuned on the test

set. And the work has to ship as a package anyone can install.

Proposed methodology. We measure the photo first and derive every setting from what we measure. Two methods then restore it. The classical pipeline is predictable and never adds detail that was not there. The Retinexformer network recovers colour and texture far better, but it sometimes invents them. A quality controller compares each output with the input and gives more weight to the one with fewer faults. We chose this design over a single fixed method because each method fails on photos the other handles well. On clean benchmark images the hybrid scores the same as the network, and when the network's output fails the controller's checks, the classical result takes over.

2 Technical architecture and implementation

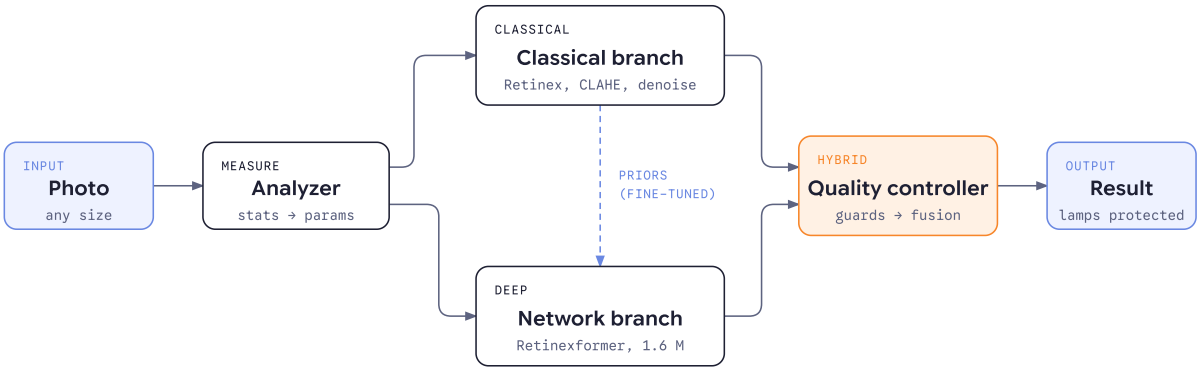


Figure 1. How `resenh.enhance` processes a photo. The analyzer's measurements set the parameters of both methods. The fine-tuned network also takes two classical outputs, the illumination map and the Retinex result, as extra input channels. The controller, highlighted, makes the final blend. The code is in `src/resenh/`: `analysis.py`, `classical/`, `models/`, `controller.py` and `pipeline.py`.

2.1 Classical algorithmic baseline

Algorithms. At each pixel we take the brightest of the three colour channels as a first estimate of the light falling there, then smooth that map with a guided filter. Dividing the image by this illumination map, raised to a power γ, brightens it in the Retinex style used by LIME (reflectance = image ÷ illumination$^{\gamma}$). Then come guided or bilateral denoising matched to the noise level, CLAHE on the brightness channel, gray-world white balance and highlight protection.

Adaptive selection logic. The analyzer measures seven statistics, and each parameter is a clipped linear function of one or two of them. We fitted the thresholds on training and validation images; the test images played no part.

Mean brightness, share of dark pixels	→ whether the photo counts as low-light; Retinex γ from 0.85 down to 0.45; minimum gain
Noise σ (Immerkaer method), relative to signal	→ denoising strength from 0 to 1; CLAHE clip from 2.5 down to 1.5
Colour cast (distance from gray)	→ white-balance strength from 0 to 0.8
Uneven lighting	→ a gentler γ, so partly lit scenes keep their depth
Share of near-white highlights	→ highlight protection, strongest for small lamps

2.2 Deep learning pipeline

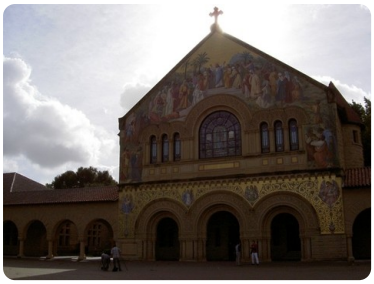
Model. Retinexformer (ICCV 2023), a Retinex-based transformer with 1.6 million parameters that restores an image in one pass. In the independent audit we followed, it had the highest plain PSNR of any model with released code on LOL-v1 and LOL-v2-Syn, and was close to the best on LOL-v2-Real. It is also small enough to run in a browser through ONNX Runtime Web: the first visit downloads about 40 MB, which the browser then caches.

Loss and training. We fine-tuned from the official checkpoints. The first layer got extra input channels for the classical outputs, with their weights set to zero, so training begins exactly at the official model. The loss is Charbonnier + 0.2·(1−SSIM) + 0.02·VGG-19 perceptual, wrapped in the GT-mean loss (ICCV 2025). Training patches grew from 128 to 256 to 384 pixels, we kept a moving average of the weights (EMA 0.999), and we varied gamma and added noise at random. Each benchmark has its own model: LOL-v1 (485 training pairs), LOL-v2-Real (689) and LOL-v2-Syn (900). We also trained a generalist model on LOL and LSRW together: 5,600 pairs once duplicates were removed. Training and validation images were split by scene, so no scene appears in both.

2.3 Hybrid integration and adaptive mechanism

Fusion strategy. The classical side guides the network at three points. Before the network, a noise gate denoises the input in proportion to the noise it measured. Inside the fine-tuned network, the illumination map and the Retinex result arrive as extra inputs. After the network, the controller compares both outputs with the input.

Decision logic. For each output the controller measures four things against the input: highlights that were not clipped before but are now, change in colour, how many edges survive, and how much the noise grew. Together they give a penalty, and each output's weight is $w \propto \text{prior} \cdot \exp(-\text{penalty} / T)$. The prior favours the network when the input is very dark. After blending, highlight protection restores lamps that the network pushed to white, and a photo that was already bright is pulled back towards the original.



Input



Network alone: **orange sky**



Classical branch



Hybrid result

Guard	Network	Classical
Newly clipped highlights	0.4%	0.0%
Colour shift vs input	0.11	0.06
Edges kept	0.47	0.67
Noise amplification	×5.4	×1.4
Fusion weight	**0%**	**100%**

Figure 2. An unseen photo from the DICM set, with the official LOL-v2-Real model as the network. The network raises the noise 5.4 times and turns the sky orange. The controller detects both and gives the classical result almost all the weight. Because the input is not very dark, the analyzer also caps the gain.

3 Experimental benchmarks and evaluation

Protocol. PSNR is measured in RGB on the saved 8-bit image, with no ground-truth brightness correction and no per-image settings. SSIM follows the MATLAB implementation, LPIPS uses AlexNet, and NIQE uses `niqe_matlab` from pyiqa. Before comparing anything, we checked that our evaluation code reproduces the official Retinexformer scores exactly: 25.15, 22.79 and 25.67 dB. The full tables, with 95% confidence intervals and the noise and edge metrics, are in `eval_P1_T46.md`.

3.1 Quantitative benchmarks

LOL-v2-Real test, 100 pairs	PSNR (dB) ↑	SSIM ↑	LPIPS ↓ / NIQE ↓	Latency, ms per MP
Classical baseline	17.44	0.724	0.417 / 4.83	382 GPU 365 CPU
Deep learning only	22.79	0.840	0.171 / 3.59	488 GPU 3774 CPU
Proposed hybrid system	**22.82**	**0.840**	**0.171 / 3.59**	**1906** GPU **4990** CPU

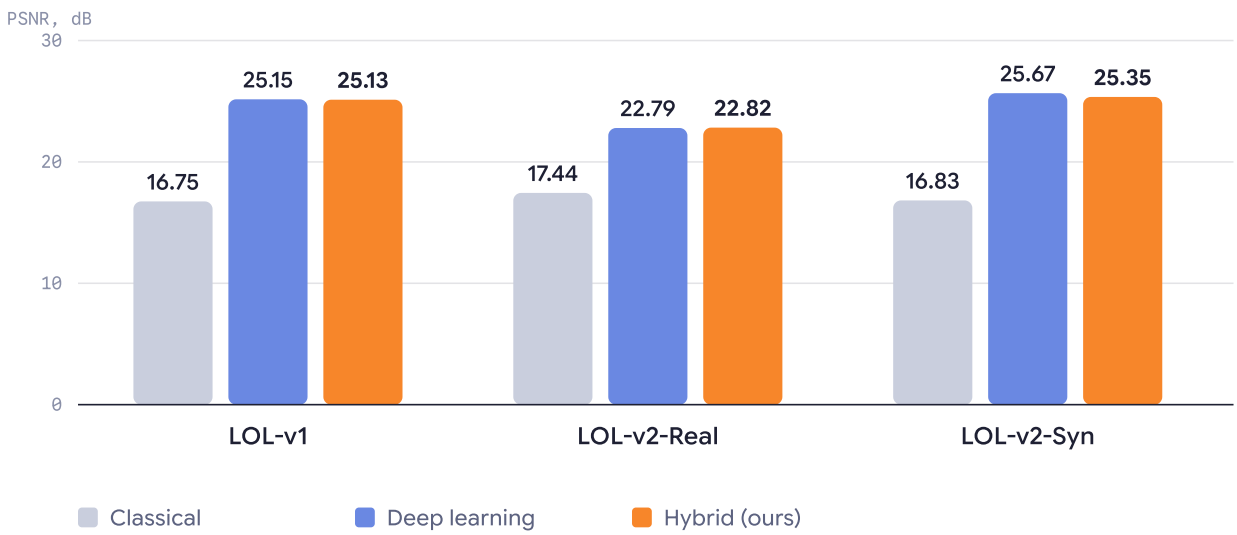


Figure 3. PSNR on the three standard test sets, of 15, 100 and 100 pairs. On every set the hybrid falls within the network's confidence interval (about ±0.7 dB on LOL-v2 and ±1.5 dB on LOL-v1). The classical method trails by 6 to 9 dB, but it never invents content, so it is safe to fall back on.

Analysis. The network supplies almost all of the accuracy. Against the classical method it adds 5 to 9 dB of PSNR and about 0.1 of SSIM, and it halves LPIPS. NIQE, which judges naturalness without a reference, ranks them the other way: on all four unseen photo sets the classical output scores better, for example 3.21 against 3.84 on DICM. The controller costs almost nothing on clean benchmark images (+0.03 dB on LOL-v2-Real). On the photo in Figure 2, it is what keeps the orange sky out of the result. On an RTX 4070 laptop GPU, a one-megapixel photo takes 0.38 s with the classical method, 0.49 s with the network alone and 1.9 s with the hybrid. The hybrid takes longest because its classical steps and priors run on the CPU. On the laptop's CPU alone the hybrid takes 5.0 s, and in a laptop browser with WebGPU about 6 s.

Unseen camera: LSRW

LSRW test, 50 pairs	PSNR	SSIM	LPIPS
Classical	16.15	0.530	0.420
Deep learning only	17.19	0.509	0.331
Hybrid, official backbone	17.32	0.515	0.334
Hybrid, general model (default)	19.26	0.584	0.361

Unseen photos, no ground truth

Set	NIQE, network	NIQE, classical
DICM	3.84	3.21
LIME	4.00	3.33
MEF	3.96	3.52
NPE	4.49	3.71

3.2 Visual outputs and intermediate representations



Input | Classical | Deep learning | **Hybrid** | Ground truth

Figure 4. A test pair from LOL-v2-Real. The classical result is bright but flat. The network and the hybrid come much closer to the colour of the reference.

`Extreme low light (LIME)`



Input | Classical | Deep learning | **Hybrid**

`Backlit scene, bright sky (VV)`



Input | Classical | Deep learning | **Hybrid**

Figure 5. Photos the models never saw. In the dark scene the shadows open up and the lamps keep their shape. In the backlit scene the sky keeps its colour and the people brighten a little, less than with the classical method.



Illumination | **Gain map** | **Retinex** | **Classical** | **Network** | **Fused result**

Figure 6. The steps behind the LIME photo: the illumination map (brightest channel, smoothed by a guided filter), the gain applied to each pixel, the Retinex result before denoising and CLAHE, the two methods' outputs, and the blend. `res.stages` returns these images, and the web demo shows them in its "Every stage" tab.

3.3 Ablation study



Figure 7. How much PSNR each component adds when switched on, with everything else unchanged. The training comparisons are 3,000-iteration fine-tunes from the same starting point. The GT-mean loss improves SSIM and LPIPS but lowers plain PSNR, so the LOL benchmark rows use the official network. For real photos the package defaults to the generalist model, which scored 1.5 dB higher on LSRW validation. The noise gate costs 0.55 dB on clean images and adds 4.3 dB on noisy ones, so the package offers two presets: `robust`, the default, and `benchmark`.

4 Deployment and quickstart

```
# 1. Install (Python 3.10+; add [dl] for the network, omit it for classical-only)
pip install "resenh[dl]"
# 2. Enhance a folder of photos
resenh enhance ./samples/ -o ./results/            # --mode hybrid | dl | classical
# 3. Or from Python, in Jupyter, Colab or Kaggle
import resenh
res = resenh.enhance("night.jpg")
res.save("night_restored.png"); print(res.summary())
# Reproduce the evaluation
git clone https://github.com/SharadhNaidu/resen && cd resen && pip install -e ".[dl,eval]"
python eval/run_eval.py --bench lolv1 lolv2_real lolv2_syn --methods classical
"net:retinexformer-{bench}" "hybrid-bench:retinexformer-{bench}" --niqe
python eval/make_report.py
```

Deployment formats. The web app, built with React and shadcn/ui, runs the same Python package inside the browser through Pyodide. The network runs in ONNX Runtime Web, on WebGPU where available and WebAssembly elsewhere. Cloudflare serves the site as static files, so there is no server to sleep or queue requests, and photos never leave the device. The Python package and command-line tool are on PyPI as `resenh`. They fetch the weights from Hugging Face on first use and check each file's SHA-256. The repository also includes a Gradio app (`resenh app`), a Dockerfile and a quickstart notebook for Colab and Kaggle.

Submission checklist

- ▣ Public GitHub repository with an updated README
- ▣ No raw code pasted into this report
- ▣ Every metric is reproducible with `eval/run_eval.py`
- ▣ Team details recorded: V-Max, T46, section E